

WSI ćwiczenie 3 - Dwuosobowe gry deterministyczne

Natasza Jarecka

2025

1 Opis algorytmu

Algorytm minimax, to algorytm przewidujący najlepszy możliwy ruch w grze deterministycznej, dwuosobowej, o sumie zerowej na podstawie obecnego stanu gry i możliwych przyszłych ruchów. W poniższym zadaniu wykorzystany został wariant z odcinaniem $\alpha - \beta$, który pozwala opuścić przeszukiwanie gałęzi drzewa decyzyjnego, które nie mają wpływu na decyzję.

Kolejne kroki algorytmu:

- Jeżeli gra jest wygrana lub głębokość przeszukiwania równa 0, zwróć ocenę obecnego stanu.
- Znajdź możliwe do wykonania ruchy.
- Jeżeli aktualnie porusza się gracz max:
 - dla każdego możliwego ruchu przypisz do wartości α wartość większą: jej obecną wartość lub wynik algorytmu dla następnych ruchów.
 - jeżeli α jest większa lub równa od β od razu zwróć wartość α .
 - Jeżeli przeszukano wszystkie gałęzie, zwróć wartość alfa.
- Jeżeli aktualnie porusza się gracz min:
 - Analogiczne postępowanie jak dla gracza max, jednak wartość przypisujemy do β , jest ona równa mniejszej wartości: aktualna wartość β lub wynik algorytmu dla następnych ruchów.
 - Zwracamy wartość β .

2 Opis gry

Gra Connect Four wykorzystuje planszę o wymiarach 7 x 6 pól. Pierwszy gracz wrzuca swój żeton do wybranej przez niego kolumny. Żeton zajmuje najniższą pozycję. Gracze wrzucają swoje żetony na przemian, aż jeden z nich ułoży cztery żetony w poziomie, pionie lub ukosie. Wygrywa ten gracz, który zrobi to jako pierwszy. Jeżeli natomiast plansza się zapełni, a nie utworzy się żadna czwórka, jest remis.

Jeżeli pierwszy gracz wykonuje optymalne ruchy, może zagwarantować sobie wygraną. Jeżeli jednak popełni błąd, gracz drugi może wygrać.

3 Implementacja

Minimax

Parametry wejściowe:

- evaluation - funkcja heurystyczna oceniająca jak dobry jest dany ruch dla obecnego gracza
- depth - głębokość przeszukiwania drzewa
- state - obiekt klasy State, zawierający informacje o jej obecnym stanie
- move_max - wartość logiczna, czy teraz ruch wykonuje max
- max_player i min_player - gracz który ma być maxem lub minem
- alpha i beta - najlepsza obecnie wypłata dla max i najlepsza dla min

Parametry wyjściowe:

- eval - ocena wybranego ruchu
- best_move - wybrany ruch

Jeśli gra się zakończyła lub osiągnięto maksymalną głębokość, funkcja zwraca ocenę aktualnego stanu oraz brak ruchu. W przeciwnym wypadku generuje wszystkie możliwe ruchy i dla każdego z nich wywołuje rekurencyjnie siebie na nowym stanie powstałym po wykonaniu ruchu. Jeśli ruch należy do gracza maksymalizującego, funkcja szuka ruchu dającego najwyższą ocenę i aktualizuje wartość alfa, przy czym przerywa pętlę, jeśli alfa przekroczy beta, co oznacza, że dalsze sprawdzanie nie ma sensu. Analogicznie, gdy ruch należy do gracza minimalizującego, szuka najniższej możliwej oceny i aktualizuje beta, również przerywając pętlę, gdy zachodzi warunek przycinania. Funkcja zwraca najlepszą ocenę oraz odpowiadający jej ruch.

Funkcja heurystyczna

Funkcja evaluate_move ocenia stan gry, zwracając liczbę całkowitą, która reprezentuje przewagę jednego z graczy. Przyjmuje jako argumenty aktualny stan gry (state) oraz dwóch graczy: maksymalizującego (max_player) i minimalizującego (min_player). Najpierw sprawdza, czy któryś z graczy wygrał, korzystając z metody get_winner(). Jeśli zwycięzcą jest gracz maksymalizujący, funkcja zwraca 1, jeśli minimalizujący – zwraca -1. W przypadku braku zwycięzcy (czyli gdy gra jeszcze trwa lub jest remis), funkcja zwraca 0.

4 Eksperymenty i wyniki

Głębokości przeszukiwań dla Max i Min	Max wygrane	Min wygrane	Remisy
4 i 4	50%	36.7%	13.3%
7 i 7	53.3%	43.3%	3.3%
4 i 7	30%	66.7%	3.3%
3 i 8	10%	90%	0%
7 i 4	70%	23.3%	6.7%
8 i 3	80%	20%	0%
losowe ruchy	55%	44.8%	0.2%

Rozpoczął gracz Max

W pojedynku przeciwko mnie, na 10 prób, gracz komputerowy będący graczem rozpoczynającym (głębokość przeszukiwania równa 8) wygrał za każdym razem. Gdy to ja byłam graczem rozpoczynającym udało mi się wygrać dwa razy.

```

Current player: 2
[ ][ ][ ][ ][ ][ ][ ]
[ ][ ][ ][ ][ ][ ][ ]
[ ][ ][ ][ ][ ][ ][ ]
[ ][ ][ ][2][ ][ ][ ]
[ ][ ][ ][2][ ][ ][ ]
[1][ ][1][2][ ][1][ ]
Current player: 1
[ ][ ][ ][ ][ ][ ][ ]
[ ][ ][ ][ ][ ][ ][ ]
[ ][ ][ ][1][ ][ ][ ]
[ ][ ][ ][2][ ][ ][ ]
[ ][ ][ ][2][ ][ ][ ]
[1][ ][1][2][ ][1][ ]

```

Gracz komputerowy blokuje możliwość wygranej (komputer - gracz 1, ja - gracz 2)

```

Current player: 2
[ ][ ][ ][ ][ ][ ][2]
[ ][ ][ ][ ][ ][ ][1]
[ ][ ][ ][1][ ][ ][2]
[ ][1][ ][2][ ][ ][2]
[ ][1][ ][2][ ][ ][2]
[1][1][ ][2][1][2][1]
Current player: 1
[ ][ ][ ][ ][ ][ ][2]
[ ][ ][ ][ ][ ][ ][1]
[ ][1][ ][1][ ][ ][2]
[ ][1][ ][2][ ][ ][2]
[ ][1][ ][2][ ][ ][2]
[1][1][ ][2][1][2][1]

```

Gracz komputerowy wykorzystuje okazję by wygrać

5 Wnioski

Algorytm zachowuje się według moich przewidywań. Przy równej głębokości przeszukiwań gracz rozpoczynający wygrywa częściej - pierwszy ruch daje większą szansę na wygraną.

Gracz z większą głębokością przeszukiwania wygrywa częściej - jest w stanie lepiej przewidzieć możliwe scenariusze rozgrywki.

Gra Connect Four jest grą rozwiązana - przy głębokości przeszukiwania 42 gracz zaczynający byłby w stanie wygrać za każdym razem. Jednak wyniki generują się za długo, bym mogła przeprowadzić taki eksperyment, muszę więc uwierzyć temu na słowo.